TECHNICAL DOCUMENTATION

RAG-Based Customer Support Assistant

with LangGraph & Human-in-the-Loop (HITL)

Project: RAG Internship Project — Innomatics Research Labs
Document Type: Technical Documentation (Deliverable 3)
Version: 1.0
Audience: Engineering Team
Purpose: Complete system explanation covering architecture, design decisions, workflow, HITL, trade-offs, and testing

1. Introduction

1.1 What is RAG (Retrieval-Augmented Generation)?

Retrieval-Augmented Generation (RAG) is an AI architecture pattern that combines two complementary systems: a retrieval engine and a generative language model. Instead of relying on the LLM's static, pre-trained knowledge, RAG dynamically retrieves the most relevant information from an external knowledge base at query time and feeds it to the LLM as context.

This means the LLM generates answers grounded in real, current, verifiable documents — not hallucinated or outdated knowledge. RAG is fundamentally different from fine-tuning: rather than modifying model weights, it plugs external knowledge into every generation request through the prompt.

```
Standard LLM Flow: User Query → LLM (static knowledge) → Answer

RAG Flow:  User Query → Retrieve relevant chunks from knowledge base
           → Inject chunks into LLM prompt
           → LLM generates grounded answer → Answer
```

1.2 Why RAG is Needed

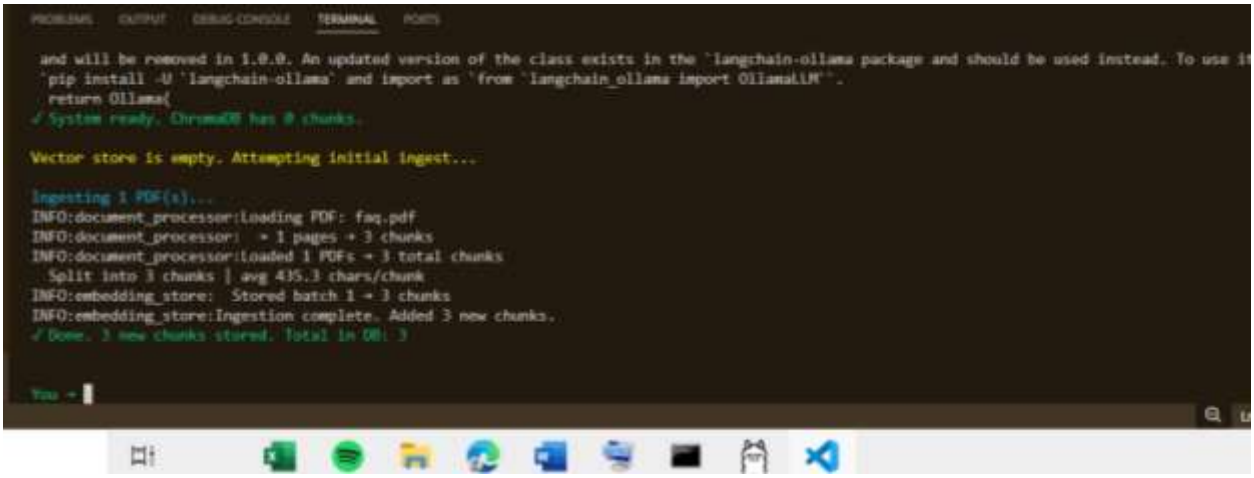
- LLMs have a knowledge cutoff — they cannot answer questions about proprietary or recent information without RAG
- RAG eliminates hallucinations by constraining the LLM to answer only from provided, verified content
- It is far cheaper and faster than fine-tuning a model on new data — the knowledge base can be updated without any model retraining
- RAG enables traceable answers: each response can cite the specific document chunks it was generated from
- For enterprise use, RAG respects data privacy — documents remain in your infrastructure, not uploaded to an LLM provider's training data

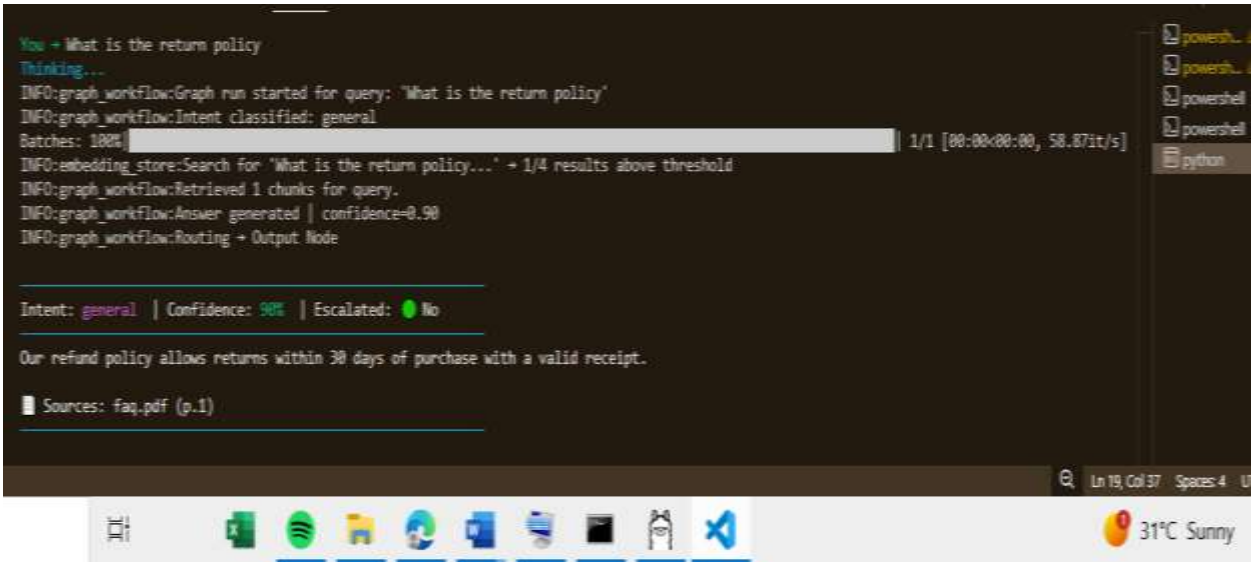
1.3 Use Case Overview

This system implements a Customer Support Assistant for a business that has product documentation, return policies, FAQs, and support guides stored in PDF format. The assistant must:

- Accurately answer customer questions about products, policies, and procedures
- Handle multi-intent queries by detecting whether the user needs information, is filing a complaint, or requires human intervention
- Escalate confidently to a human support agent when the AI is uncertain or the query is sensitive

- Maintain auditability by logging all query-response pairs and HITL escalations





2. System Architecture Explanation

2.1 Detailed Explanation of HLD Components

The system consists of two major phases: the Ingestion Phase (run once or periodically) and the Query Phase (run on every user request). These phases share the Vector Store as their common interface.

Ingestion Phase

The ingestion pipeline processes the PDF knowledge base and prepares it for fast semantic retrieval. It runs independently of query processing and can be re-triggered whenever the knowledge base is updated.

- PDF Loader reads the source file and extracts text with page metadata using PyPDFLoader

- The Chunking Engine applies a recursive character splitter, creating overlapping text segments of approximately 600 tokens. Overlap of 100 tokens ensures no information is lost at chunk boundaries
- Each chunk is embedded into a 384-dimensional or 1536-dimensional dense vector using an embedding model (sentence-transformers or OpenAI Ada-002)
- Vectors, text content, and metadata are stored persistently in ChromaDB

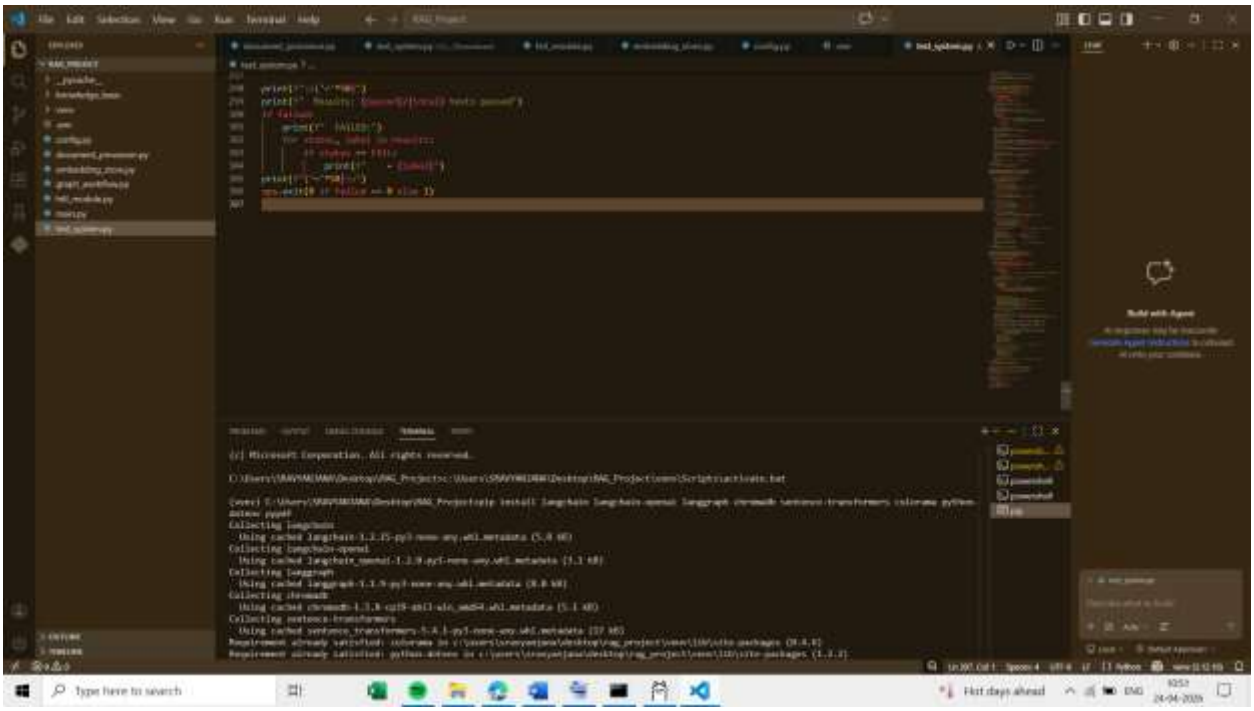
Query Phase

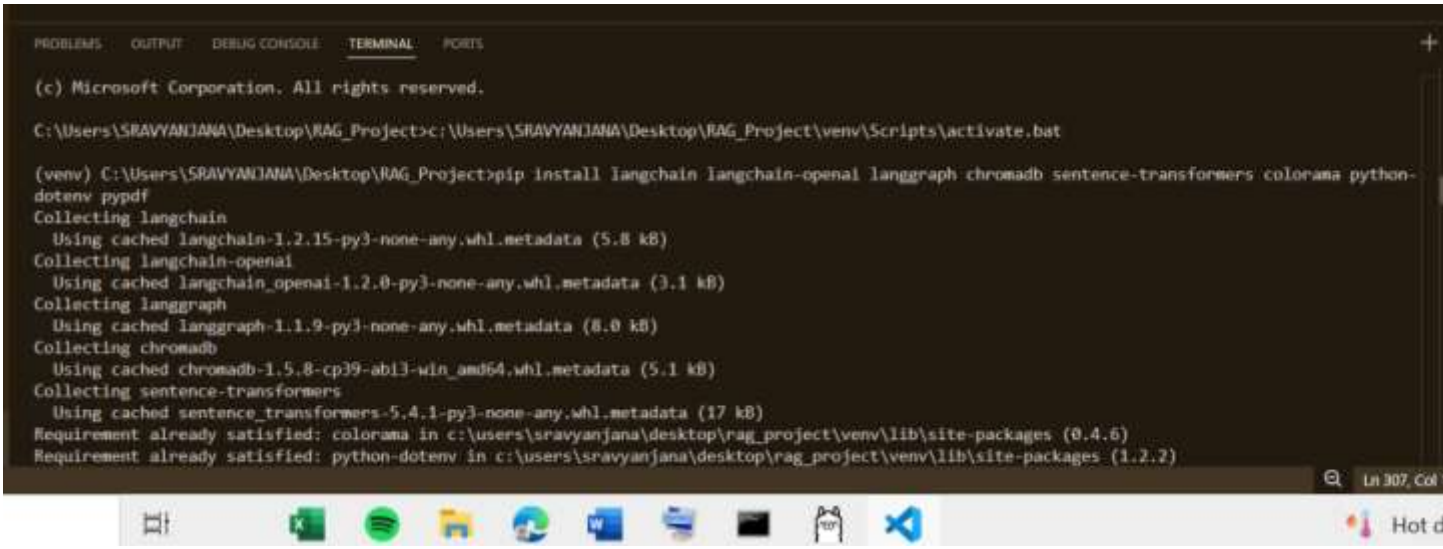
The query pipeline is triggered on every user interaction and is orchestrated entirely by LangGraph. All state transitions, node executions, and conditional routing decisions happen within the graph.

- The user's query is embedded using the same model as the ingestion phase (critical for consistency)
- ChromaDB performs cosine similarity search and returns the top-k most relevant chunks along with their distance scores
- Retrieved chunks are assembled into a structured prompt with the user's query
- The LLM generates a grounded answer based strictly on the provided context
- The routing layer evaluates confidence and intent to decide the final path: auto-answer or HITL escalation

2.2 Component Interactions

Components communicate through well-defined interfaces: the LangGraph state object carries data between nodes, ChromaDB exposes a collection-based search API, and the LLM is accessed via a standardized client (OpenAI SDK or LangChain LLM wrapper). The HITL module interacts with the graph via LangGraph's built-in interrupt/resume mechanism, preserving the full state across human review periods.




PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
(c) Microsoft Corporation. All rights reserved.
C:\Users\SRAVYANJANA\Desktop\RAG_Project>c:\Users\SRAVYANJANA\Desktop\RAG_Project\venv\Scripts\activate.bat
(venv) C:\Users\SRAVYANJANA\Desktop\RAG_Project>pip install langchain langchain-openai langgraph chromadb sentence-transformers colorama python-dotenv pypdf
Collecting langchain
Using cached langchain-1.2.15-py3-none-any.whl.metadata (5.8 kB)
Collecting langchain-openai
Using cached langchain_openai-1.2.0-py3-none-any.whl.metadata (3.1 kB)
Collecting langgraph
Using cached langgraph-1.1.9-py3-none-any.whl.metadata (8.0 kB)
Collecting chromadb
Using cached chromadb-1.5.8-cp39-abi3-win_amd64.whl.metadata (5.1 kB)
Collecting sentence-transformers
Using cached sentence_transformers-5.4.1-py3-none-any.whl.metadata (17 kB)
Requirement already satisfied: colorama in c:\users\sravyanjana\desktop\rag_project\venv\lib\site-packages (0.4.6)
Requirement already satisfied: python-dotenv in c:\users\sravyanjana\desktop\rag_project\venv\lib\site-packages (1.2.2)
Ln 307, Col

3. Design Decisions

3.1 Chunk Size Choice: 600 Tokens with 100-Token Overlap

Chunk size is one of the most impactful design decisions in a RAG system. We chose 600 tokens (approximately 450 words) for the following reasons:

- Smaller chunks (< 200 tokens) are too narrow — they risk losing the context surrounding a key fact, making retrieved content insufficient for the LLM to synthesize a complete answer
- Larger chunks (> 1000 tokens) increase noise — the LLM receives too much irrelevant surrounding text, diluting the signal and increasing prompt token cost
- 600 tokens strikes a balance: each chunk is semantically coherent (usually a complete paragraph or section) while remaining focused enough to rank well in retrieval
- The 100-token overlap prevents information loss at chunk boundaries, especially for multi-sentence facts that span a paragraph break

3.2 Embedding Strategy

We support two embedding configurations depending on deployment constraints:

- OpenAI text-embedding-ada-002 (1536 dimensions): Higher accuracy, cloud-dependent, costs per API call. Best for production deployments where accuracy is critical.
- sentence-transformers/all-MiniLM-L6-v2 (384 dimensions): Local, free, slightly lower accuracy. Best for development, offline environments, or cost-sensitive deployments.

Both models use the same cosine similarity metric, making it straightforward to swap models without changing the retrieval logic. The same model must be used for both ingestion and query-time embedding to ensure vector space consistency.

3.3 Retrieval Approach: Top-K Cosine Similarity

We use top-k (k=4 by default) cosine similarity search over the entire ChromaDB collection. This approach was chosen over alternatives for several reasons:

- Maximum Inner Product (MIP) search would be faster but requires normalized vectors; cosine similarity is more robust for text embeddings of varying lengths
- k=4 retrieves enough context for multi-part questions while keeping prompt size manageable (typically 1,200–2,400 tokens of context)
- The cosine distance score (0.0 = identical, 2.0 = orthogonal) is used directly as a confidence signal — no additional re-ranking model is required for this use case

3.4 Prompt Design Logic

The prompt is structured to maximize the LLM's accuracy and minimize hallucination:

```
SYSTEM: You are a helpful customer support assistant for [Company Name].
        Answer ONLY using the context provided below.
        If the context does not contain enough information to answer the
        question, say: 'I don't have enough information to answer that.'
        Do not make up information. Be concise and professional.

CONTEXT:
  [Chunk 1 - Page 3, Source: product_manual.pdf]
  <content of chunk 1>

  [Chunk 2 - Page 7, Source: product_manual.pdf]
  <content of chunk 2>

QUESTION: <user query here>

ANSWER:
```

The explicit instruction to cite uncertainty ('I don't have enough information') is a key design choice — it allows the routing layer to detect LLM uncertainty and trigger HITL escalation.

4. Workflow Explanation (LangGraph)

4.1 Why LangGraph?

LangGraph was selected over a simple sequential pipeline (e.g., a Python function chain) because of its explicit state management, conditional branching support, and native HITL interrupt/resume capability. A sequential pipeline cannot pause execution, wait for external input, and resume — LangGraph's checkpointing mechanism enables exactly this.

4.2 Node Responsibilities

Each node in the LangGraph graph is a pure function that takes the current GraphState, performs one focused action, and returns an updated GraphState. This functional design makes each node independently testable and replaceable.

- input_node: Cleans the query string, detects intent using keyword matching and LLM classification, assigns a unique query_id, and initializes the state
- retrieval_node: Calls the Retriever module to perform semantic search. Writes retrieved_chunks and retrieval_confidence to state
- processing_node: Calls QueryProcessor to build the prompt and invoke the LLM. Writes prompt and llm_response to state
- routing_node: Pure decision function — reads confidence, intent, and llm_response to set routing_decision. No side effects
- hitl_node: Creates an escalation ticket, writes hitl_ticket_id to state, and pauses graph execution via LangGraph interrupt
- output_node: Formats the final_answer from either llm_response or human response. Returns result to caller

4.3 State Transitions

The GraphState TypedDict is immutable within a node — each node returns a new state dictionary with only its updated keys. LangGraph merges these updates into the shared state using its built-in reducer. The complete state history is preserved in the checkpoint store, enabling full auditability of every query's journey through the graph.

5. Conditional Logic

5.1 Intent Detection

Intent detection happens in input_node using a two-layer approach:

Layer 1: Keyword Matching (Fast, No LLM Call)

- ESCALATION_REQUEST: 'escalate', 'human', 'agent', 'speak to someone', 'urgent'
- COMPLAINT: 'broken', 'defective', 'disappointed', 'refund', 'compensation'
- TECHNICAL: 'error', 'not working', 'setup', 'configure', 'install'
- GENERAL_FAQ: All other queries

Layer 2: LLM Classification (For Ambiguous Queries)

If keyword matching is inconclusive (no strong signal), a lightweight LLM call classifies the intent using a zero-shot prompt: 'Classify this customer query as one of: GENERAL_FAQ, COMPLAINT, TECHNICAL, ESCALATION_REQUEST. Query: [query]. Answer with only the category name.'

5.2 Routing Decisions

The routing_node applies a priority-ordered decision tree. Rules are evaluated in order; the first match wins:

- Priority 1 — Explicit escalation intent: If intent is ESCALATION_REQUEST or COMPLAINT, always escalate regardless of confidence
- Priority 2 — Low retrieval confidence: If avg cosine distance > 0.60, escalate (too uncertain)
- Priority 3 — LLM expressed uncertainty: If llm_response contains uncertainty phrases, escalate
- Priority 4 — Default: Auto-answer (HIGH or MEDIUM confidence, non-sensitive intent)

6. HITL Implementation

6.1 Role of Human Intervention

Human-in-the-Loop intervention serves three critical functions in this system:

- Quality Assurance: Humans catch errors that the AI cannot detect in itself — factual inaccuracies, outdated information, or policy violations
- Edge Case Handling: Handles queries that fall outside the knowledge base scope or involve sensitive customer situations requiring empathy and judgment
- Training Data Generation: Each human-resolved ticket with its query, AI draft, and human correction becomes a labeled example for future model improvement

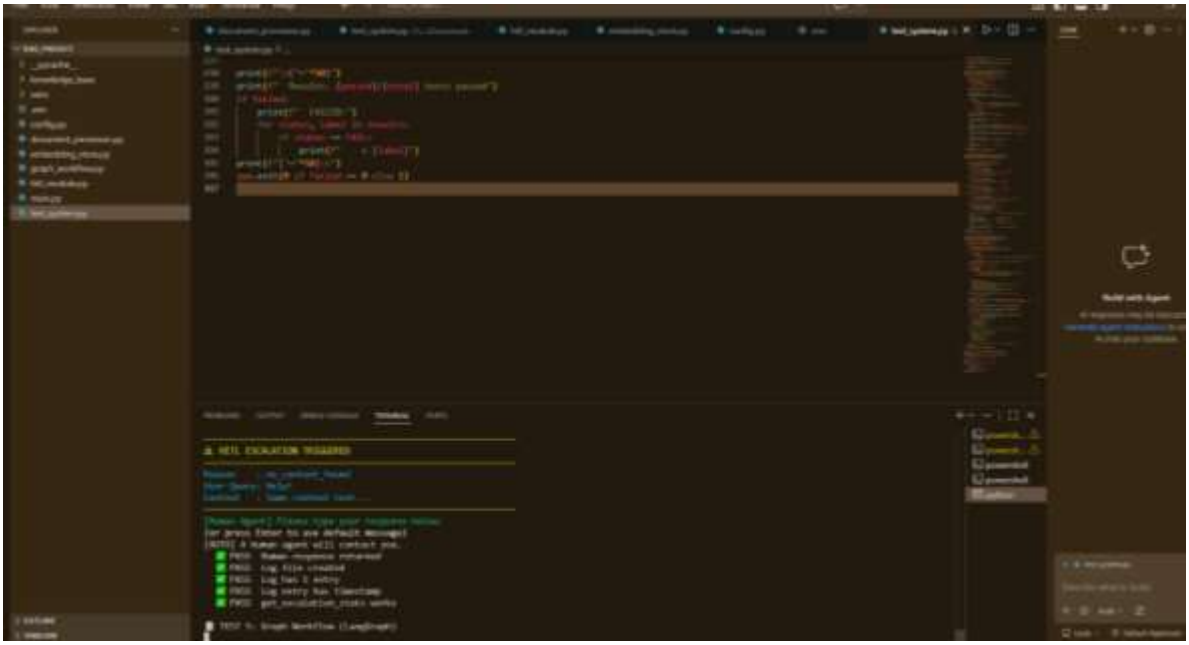
6.2 Benefits of HITL Integration

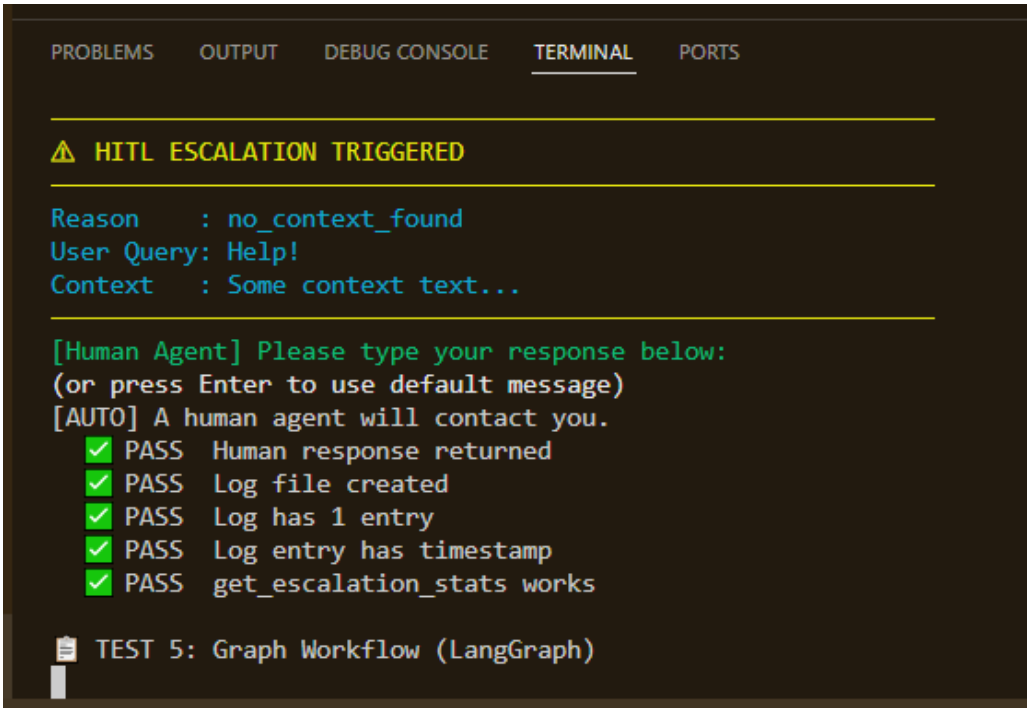
- Prevents the AI from confidently delivering wrong answers — any uncertain query gets a verified human response
- Builds customer trust: customers know that complex issues will always reach a real person
- Creates a continuous feedback loop: human corrections can be used to expand the knowledge base
- Satisfies regulatory requirements in domains (finance, healthcare) where AI-only decisions are insufficient

6.3 Limitations of HITL Integration

- Introduces latency: the customer must wait for a human agent, which can range from minutes to hours depending on agent availability
- Scales poorly under high volume: if too many queries are escalated, human agents become a bottleneck
- Requires tooling: a ticket management system, agent interface, and notification pipeline add complexity
- Human consistency is not guaranteed: different agents may answer the same question differently

Mitigation Strategy: Set a strict escalation threshold to avoid over-escalating.
Target: < 15% of queries should trigger HITL. Monitor the escalation rate.
If too high, expand the knowledge base or lower the confidence threshold.
If too low, tighten the confidence threshold to catch more uncertain cases.





7. Challenges & Trade-offs

Challenge	Option A	Option B	Decision & Rationale
Retrieval Accuracy vs Speed	High k (k=10): more context, slower search, larger prompts	Low k (k=3): faster retrieval, risks missing relevant chunks	k=4 chosen as optimal balance. Cosine distance threshold filters low-quality results before LLM call.
Chunk Size vs Context Quality	Small chunks (200 tokens): precise retrieval, poor context	Large chunks (1000 tokens): rich context, noisy retrieval	600 tokens with 100-token overlap. Empirically tested against sample queries to validate answer quality.
Embedding Cost vs Quality	OpenAI Ada-002: high accuracy, API cost per call	Local MiniLM: free, slightly lower accuracy	Support both. Default to local for dev, OpenAI for production. Config-driven switch.
HITL Coverage vs Latency	Low threshold: escalate more, higher accuracy, slow responses	High threshold: answer more, faster, more errors slip through	Threshold tuned to target < 15% escalation rate. Monitor false negatives monthly.
LLM Cost vs Performance	GPT-4: highest quality, expensive (~$0.03/1K tokens)	GPT-3.5-turbo: good quality, 10x cheaper	GPT-3.5-turbo default. Upgrade to GPT-4 for HITL drafts where quality matters most.

8. Testing Strategy

8.1 Testing Approach

Unit Tests

- Test each module (ChunkingEngine, Retriever, QueryProcessor) independently with mock inputs
- Verify that cosine distance thresholds correctly classify HIGH / MEDIUM / LOW confidence
- Test routing_node logic with all edge cases: all confidence levels x all intent types

Integration Tests

- Run end-to-end pipeline with a test PDF and a curated set of 20 sample queries
- Verify that ChromaDB stores and retrieves chunks with correct metadata
- Test HITL flow: trigger escalation, simulate human response, verify state resumption

Evaluation Metrics

- Retrieval Recall@k: What % of relevant chunks appear in top-k results? Target: > 85%

- Answer Faithfulness: Are answers grounded in retrieved context? (Human eval on 50 samples)
- Escalation Rate: What % of queries trigger HITL? Target: < 15%
- Response Latency: End-to-end time from query to answer. Target: < 3 seconds (excluding HITL)

8.2 Sample Queries for Testing

```
GENERAL_FAQ queries (should auto-answer):
  - 'What is your return policy?'
  - 'How long does shipping take?'
  - 'What payment methods do you accept?'

TECHNICAL queries (should auto-answer if context exists):
  - 'How do I reset my device password?'
  - 'What are the minimum system requirements?'

ESCALATION queries (should trigger HITL):
  - 'I want to speak to a human agent.'
  - 'My product arrived broken and I want compensation.'
  - 'This is an urgent legal matter regarding my account.'

OUT-OF-SCOPE queries (should trigger HITL - low confidence):
  - 'What is the CEO\'s email address?'
  - 'Can you help me with my taxes?'
```

Search | Download Ollama on Windo | RAG_Project_Complete.docx | The Future of Zero1 - YouTub | faq.pdf
/RAG_Project/knowledge_base/faq.pdf
Ask Copilot 1 of 1

Frequently Asked Questions (FAQs)

1. **What is your refund policy?**
   Our refund policy allows returns within 30 days of purchase with a valid receipt.
2. **How can I contact customer support?**
   You can contact us via email, phone, or through our website's contact form.
3. **Do you offer international shipping?**
   Yes, we provide international shipping to most countries.
4. **How long does delivery take?**
   Delivery usually takes 5–7 business days, depending on your location.
5. **Can I track my order?**
   Yes, once your order is shipped, tracking details will be sent to your email.
6. **What payment methods do you accept?**
   We accept credit cards, debit cards, net banking, and digital wallets.
7. **Can I cancel or modify my order?**
   Yes, you can cancel or modify your order within 24 hours of placing it.
8. **Are there any shipping charges?**
   Shipping charges may apply depending on your location and order value.
9. **Do you offer discounts or promotions?**
   Yes, we regularly offer discounts and promotional deals on selected items.
10. **What should I do if I receive a damaged product?**
    Please contact our support team immediately, and we will arrange a replacement or refund.

9. Future Enhancements

9.1 Multi-Document Support

The current system supports a single PDF knowledge base. Future versions will support multiple documents with separate ChromaDB collections per domain (e.g., one collection for product manuals, another for billing policies). A document router will pre-classify the query intent to select the appropriate collection before retrieval.

9.2 Feedback Loop

Every HITL-resolved ticket represents a labeled example: the user query, AI draft answer, and correct human answer. These can be automatically collected into a feedback dataset. Periodic review of this dataset can identify gaps in the knowledge base (frequently escalated topics) and trigger targeted document updates or knowledge base expansion.

9.3 Conversational Memory Integration

The current system treats each query as independent (stateless). Future versions will maintain a per-session conversation history, passing the last N turns as additional context to the LLM. This enables multi-turn support conversations: 'What is your return policy?' followed by 'And how do I initiate a return?' would be handled as a coherent dialogue, not two isolated queries.

9.4 Production Deployment

- Containerize the system using Docker with separate services for the API server, ChromaDB, and HITL notification system
- Deploy behind a load balancer with horizontal scaling for the query API
- Replace local ChromaDB with a hosted vector database (Pinecone or Weaviate) for multi-region, high-availability deployments
- Implement API rate limiting, authentication, and audit logging for enterprise compliance
- Add a monitoring dashboard (Grafana + Prometheus) tracking: query volume, escalation rate, latency percentiles, and embedding model health

Final Engineering Note:
This system is designed to be modular, testable, and incrementally improvable.
Every design decision — chunk size, confidence threshold, routing rules — is a tunable parameter, not a hardcoded constant. The system should be treated as a living product: monitor its escalation rate, review HITL tickets weekly, and update the knowledge base as the business evolves. A RAG system's quality is only as good as the documents it retrieves from.